Collaborative Research: CSR: Cross-Layer Context-Aware System Design for Scalable and Efficient Graph-based Retrieval-Augmented Generation

Overview

Large language models (LLMs) are rapidly evolving into agentic AI systems with reasoning capabilities that interact with external knowledge and data. In this emerging paradigm, retrieval is becoming a fundamental interface through which foundation models access external knowledge. **Graph-based retrieval-augmented generation (GraphRAG)** further organizes this knowledge around entities, relations, and multi-hop reasoning structures, enabling complex reasoning while exposing rich semantic information before LLM execution. However, GraphRAG also introduces substantial systems challenges: graph traversal grows rapidly with reasoning depth, retrieved contexts increase KV-cache and memory demand, and recurring semantic structures cause redundant retrieval, prefill computation, and data movement. More importantly, today's computing systems do not systematically exploit retrieval-derived information to guide downstream execution. This collaborative project will develop a **cross-layer, retrieval-informed system framework for scalable and efficient GraphRAG**. The central hypothesis is that semantic and structural information exposed during retrieval can guide computation, communication, and memory management across the computing stack. The project will pursue four tightly integrated research thrusts: (1) unified graph abstractions and efficient retrieval algorithms that preserve reusable semantic structures and expose execution metadata; (2) retrieval-context-aware memory optimization that uses semantic lookahead and early execution signals to guide expert placement, numerical precision, expert-cache management, and semantic-path KV-cache reuse; (3) context-aware system and hardware optimization that coordinates selective data movement and execution scheduling while accelerating graph retrieval through near-memory hardware support; and (4) benchmarking, simulation, and application-validation infrastructure for end-to-end evaluation.

Intellectual Merit

The intellectual merit of this project lies in establishing **retrieval semantics as a new systems abstraction for retrieval-augmented AI systems**. This project will investigate how retrieval information can be preserved and propagated across system layers to enable retrieval-informed cross-layer optimization. First, it will develop representations that bridge graph semantics and system requirements, allowing reusable reasoning structures to remain identifiable across retrieval and generation. Second, it will develop mechanisms that use retrieval-derived context together with runtime signals to guide heterogeneous computation, communication, caching, and memory-resource allocation. Third, it will exploit semantic locality, evidence importance, and cross-query reuse to reduce redundant graph processing, KV-cache computation, data movement, and generation overhead. These advances will establish new principles for jointly designing retrieval algorithms, runtime systems, memory hierarchies, and LLM execution, providing a systems foundation for scalable retrieval-augmented and agentic AI.

Broader Impacts

The success of this timely work will have broad impacts on the design of future retrieval-augmented and agentic AI systems, spanning retrieval algorithms, runtime systems, memory hierarchies, and computer architectures. It will provide an essential understanding of how retrieval-derived semantic and structural information can be exploited to improve the efficiency and scalability of Graph-based RAG systems. The PIs will develop courses and student projects at the intersection of computer architecture, AI systems, graph processing, and retrieval-augmented generation, helping bridge the knowledge gap across these areas. All research findings, benchmarks, software, and simulation tools will be disseminated to the community through conference and journal publications, professional meetings, open-source releases, and a dedicated project website. The PIs are committed to expanding outreach, education, training, and broadening participation in computing through this project by engaging and training graduate, undergraduate, and K–12 students in emerging AI and computer systems research. The proposed application studies in hardware code generation and biomedical knowledge reasoning will further demonstrate the potential of the developed techniques for AI-assisted engineering and scientific discovery. All outreach and similar activities will be aligned with NSF priorities and available to all Americans.

Collaborative Research: CSR: Cross-Layer Context-Aware System Design for Scalable and Efficient Graph-based Retrieval-Augmented Generation

1. Introduction

Large language models (LLMs) are rapidly evolving from stand-alone generative models into reasoning and agentic AI systems that interact with external knowledge, tools, and memory [1–4]. In this paradigm, retrieval is becoming a fundamental interface through which foundation models access external knowledge, determining what information and context are available for reasoning and subsequent actions. As agentic AI and reasoning increasingly rely on repeated retrieval across multi-step workflows, system efficiency will depend not only on LLM execution, but also on how retrieved knowledge is discovered across the computing stack. Graph-based retrieval-augmented generation (GraphRAG) [5–9] advances this new class of AI systems by organizing external knowledge around entities, relations, and multi-hop reasoning structures rather than independent text chunks. This structured retrieval enables explicit relationship discovery and multi-hop reasoning over complex knowledge.

However, in today's computing systems, retrieval and LLM execution are typically designed and optimized as separate stages [4, 10–17]: the retrieval system determines *what information* should be provided to the model, while the generation stage subsequently determines *how the resulting tokens* should be executed. Once retrieved knowledge is serialized into a prompt, much of its semantic and structural information is lost to the underlying system. This limitation becomes increasingly costly as retrieval-augmented LLM systems scale. For example, multi-hop graph reasoning expands the retrieval search space and increases the amount of evidence injected into LLM contexts, leading to growing prefill computation, KV-cache capacity, memory traffic, and data movement [6, 8, 10, 18–24].

Our Vision: This project is based on a fundamentally different perspective: retrieval is not merely a mechanism for selecting information—it can serve as an early semantic lookahead signal for the entire computing system. Before LLM generation begins, GraphRAG already reveals which entities and relations are relevant, which reasoning paths are likely to be consumed, how large the resulting context will be, which evidence is shared across queries, and how different pieces of information contribute to the answer. We hypothesize that preserving and propagating this information across the computing stack will enable improved optimization of computation, communication, and memory efficiency.

Goal and Objectives: To capitalize on this opportunity, George Washington University (GWU), the University of Central Florida (UCF), and North Dakota State University (NDSU) propose a **cross-layer context-aware system framework for scalable and efficient Graph-based RAG**. The central idea of the project is that **retrieval semantics can be preserved and propagated across system layers to guide efficient optimization of computation, communication, and memory resources**. Rather than treating graph retrieval and LLM generation as separate optimization problems, the proposed framework will develop a unified system design that uses retrieval-derived information as prior knowledge to guide KV-cache management, generation computation, and memory-system optimization. This represents a shift from conventional retrieval-agnostic system optimization toward retrieval-informed, context-aware system design.

The project will pursue four tightly integrated research thrusts as shown in Fig. 1. First, we will develop unified graph abstractions and efficient retrieval algorithms that preserve the semantic identity of retrieved evidence while exposing execution metadata and opportunities for reuse across queries. Second, we will develop retrieval-context-aware optimization techniques that leverage semantic lookahead and early execution signals to guide quantization, data placement, caching, and KV-cache management across CPUs, GPUs, and heterogeneous memory resources. Third, we will develop context-aware architectural mechanisms that exploit retrieval information to coordinate computation, data movement, and KV-cache placement, together with in-memory acceleration architectures tailored to the irregular and data-intensive characteristics of graph-based retrieval. Finally, we will establish reusable GraphRAG benchmarking, architectural simulation, and system-prototyping infrastructure, and evaluate the integrated framework using representative multi-hop and conversational workloads, as well as application test cases in **hardware code**

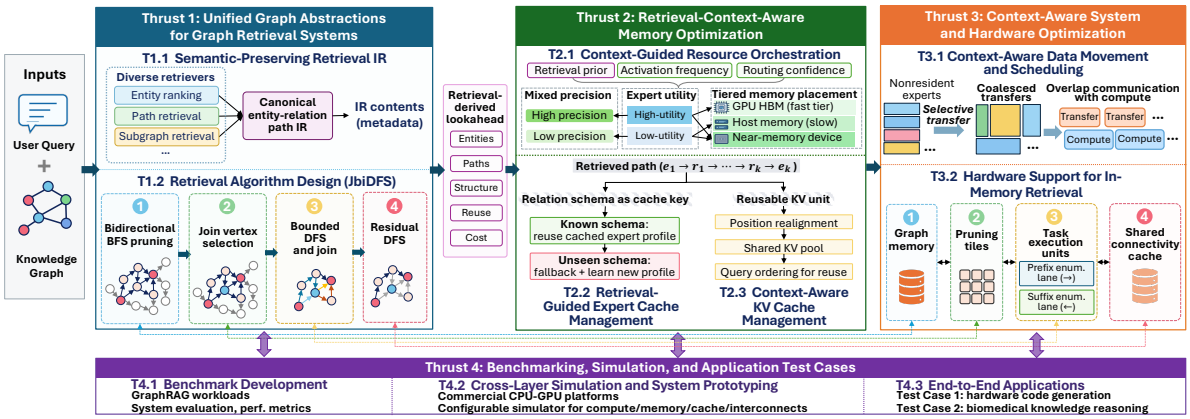


Figure 1: Overview of the proposed system.

generation and biomedical knowledge reasoning. Collectively, these four thrusts span retrieval, runtime, architecture, memory, and generation, enabling cross-layer optimization that cannot be achieved through isolated improvements at individual layers.

The expected outcome is a new system foundation for retrieval-augmented AI in which **knowledge retrieved for reasoning simultaneously informs how the underlying computing system executes that reasoning**. By exposing future computational needs earlier and exploiting semantic locality across queries and execution stages, the proposed research seeks to reduce redundant computation, memory footprint, data movement, and energy consumption while improving the scalability and throughput of emerging GraphRAG systems. More broadly, the project will establish principles for using application-level semantic information as a first-class systems signal, providing a pathway toward future computing systems that more tightly integrate AI algorithms, runtime software, memory systems, and computer architecture.

1.1. Team Qualification

The research team brings complementary expertise in computer architecture, machine learning, and memory systems, providing the cross-layer knowledge required to address the algorithmic and systems challenges of scalable and efficient Graph-based Retrieval-Augmented Generation. The PIs' research spans leading venues in computer architecture, electronic design automation, and machine learning, including ISCA, MICRO, HPCA, DAC, TCAD, and NeurIPS. PI Nan Wu (GWU) works at the intersection of computer architecture and machine learning, with particular expertise in graph learning and ML for computer systems. PI Sumitha George (NDSU) brings complementary expertise in VLSI design, computer architecture, memory hierarchies, and emerging memory technologies, with prior research on specialized cache and memory architectures for reducing data movement in data-intensive workloads. PI Hao Zheng (UCF) has expertise in computer architecture and AI/ML systems, including domain-specific acceleration, communication systems, and architectural support for graph computations. Together, the three PIs provide expertise spanning the entire computing stack—from graph-based retrieval and context construction to runtime optimization, memory hierarchy, and LLM serving—enabling tightly integrated cross-layer optimization.

2. Motivation and Preliminary Results

GraphRAG enhances retrieval-augmented generation by organizing evidence around entities, relations, and multi-hop reasoning rather than treating text chunks independently. This can improve evidence coverage and provenance, but it also introduces new challenges and opportunities for modern system design. In particular, the retrieval stage exposes rich contextual information that can potentially be exploited to enable lookahead and context-aware system optimizations. To characterize these cross-stage algorithmic and architectural challenges and opportunities, we conducted a preliminary study on an NVIDIA A100-80GB GPU using RoG-CWQ (CWQ), RoG-WEBQSP (WEB), and MetaQA-3hop (MQA), covering heterogeneous Freebase-based graphs as well as knowledge graphs [6, 25].

Semantic Locality Creates Repeated Graph Work: Real-world queries are neither independent nor uniformly distributed. Across users, accesses are often concentrated on popular entities, while successive queries within the same session tend to remain semantically related. Fig. 2(a) shows that the access frequencies of entities (e.g., tokens, graph entities) in MS MARCO closely follow a Zipfian distribution (R^2=0.994), with the 1,000 most frequently accessed entities accounting for 36.6% of all accesses [26]. Fig. 2(b) reveals strong temporal semantic locality in TREC CAsT, where consecutive conversational turns have a mean cosine similarity of 0.66 [27]. In GraphRAG, such semantic locality can translate into graph locality: related queries frequently revisit common entities and traverse overlapping entity–relation paths. Our preliminary measurements confirm this behavior under entity-clustered batching. On CWQ, 37% of retrieved-path prefill computations are redundant, with token-level redundancy differing by less than two percentage points, and 52% of queries sharing at least one previously encountered path. Even WEB, which exhibits lower overlap, shows 15% redundant path-prefill computation, with 28% of queries containing at least one recurring path. *These results suggest that redundant graph traversal and repeated LLM prefill are systematic characteristics of GraphRAG workloads rather than occasional duplication.*

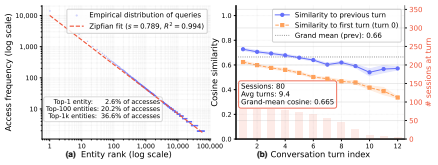


Figure 2: Semantic locality in real query workloads: (a) entity popularity in MS MARCO and (b) query similarity across TREC CAsT conversation turns.

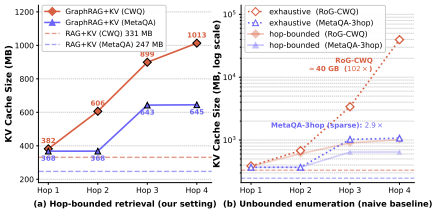


Figure 3: GraphRAG KV-cache growth with reasoning depth: (a) bounded retrieval and (b) exhaustive path enumeration.

Computation and Memory Overhead of Multi-Hop Reasoning: GraphRAG's reasoning advantage depends on retrieving paths beyond the immediate neighborhood, yet each additional hop expands the traversal space and injects more evidence into the LLM prompt. Existing systems therefore impose both a hop limit and a path budget. For example, RoG retains only paths matching its predicted relation plans—37 paths per CWQ query at four hops—rather than enumerating every simple path [6]. Even under this bound, Fig. 3(a) shows that the KV-cache footprint from one to four hops grows by 2.7× on CWQ and 1.8× on MQA, reaching 3.1× and 2.6× the corresponding text-RAG footprint. Removing the path budget exposes the underlying combinatorial growth: Fig. 3(b) reaches approximately 40 GB on dense CWQ at four hops, a 102× increase, while the sparser MQA graph grows by 2.9×. This depends strongly on graph topology and leaves less accelerator memory for model weights, activations, and concurrent requests. The same trend appears in execution time. In our preliminary study, CWQ path enumeration grows from 82 paths and 0.20 ms at one hop to 4,826 paths and 1,139 ms at four hops; MQA grows from 70 paths and 0.05 ms to 2,380 paths and 101 ms. With conventional decode-phase KV caching enabled, retrieval consequently grows to 53.6% of CWQ end-to-end latency at deeper hops, and four-hop GraphRAG achieves only 8.7% of the throughput of text RAG. Simply retrieving without a bound is not a solution: simple-path enumeration is computationally intractable at scale [28], long noisy contexts are used poorly by LLMs [29, 30], and KGQA accuracy can degrade when the retrieved subgraph becomes too large [31]. *The core requirement is therefore not unrestricted retrieval, but efficient and reusable discovery of answer-relevant multi-hop evidence.*

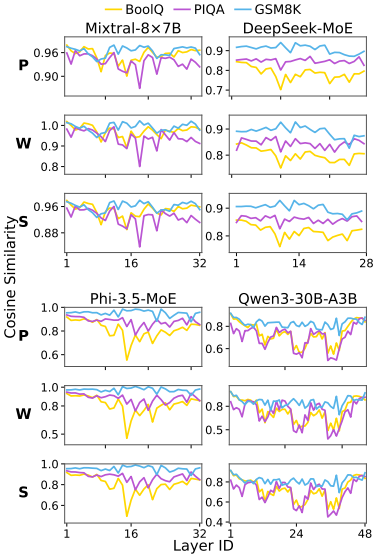


Figure 4: Prefill–decoding cosine similarity across layers for three expert-level distributions.

Early Context Signals Predict Downstream Resource Demand: Graph-based RAG exposes semantic structure before generation, but exploiting this information requires downstream execution demand to be sufficiently predictable. Fig. 4 shows the prefill–decoding cosine similarity of resource activation frequency

P, contribution strength W, and their combined importance score S across four MoE models [32–35] and three representative tasks. The average similarities are approximately 0.85, 0.83, and 0.84, respectively, indicating that early execution behavior remains highly correlated with subsequent decoding demand. *These results suggest that retrieval-derived context, combined with early execution signals, can provide reliable lookahead for proactive system optimization across subsequent generation.*

Existing Caches Miss Graph-Semantic Reuse: Modern LLM serving systems cache KV states for identical prefixes, reusable text chunks, or selected tokens [10, 11, 36–39]. These mechanisms operate on serialized text and generally assume a stable prefix, token order, or chunk boundary. GraphRAG violates these assumptions: the same entity–relation path can recur at a different prompt position, in a different path order, or under a different query context. Its keys are position-dependent, and its values can be context-sensitive. Reusing every repeated text fragment may therefore corrupt attention, while recomputing every occurrence discards the structural locality revealed before generation. Conventional retrieval optimizations are also tied to a particular index, ranking rule, or traversal strategy and do not expose reusable semantic units to the LLM runtime. *What is missing is a representation that preserves graph identity across the retrieval–generation boundary while making position, context, size, relevance, and expected reuse visible to the system.*

3. Research Plan

3.1. Research Thrust 1: Unified Graph Abstractions for Graph Retrieval Systems

Current GraphRAG systems primarily represent retrieval relevance and evidence, but expose little information about downstream execution, such as reuse opportunities, computation cost, KV-cache demand, or approximation sensitivity. This disconnect prevents the runtime from using retrieval semantics to coordinate caching, prefetching, placement, and compression across retrieval and generation. In this thrust, we will develop (1) a unified GraphRAG representation that propagates retrieval semantics and execution metadata across the system stack, and (2) an efficient retrieval algorithm that eliminates redundant graph traversal while preserving retrieval completeness.

Task 1.1: Semantic-Preserving Retrieval IR for GraphRAG

GraphRAG systems differ in how they retrieve and represent evidence: entity-ranking methods [7, 40] prioritize nodes or relations; path-based methods [6] construct multi-hop paths; graph-propagation methods [9] exploit graph connectivity; and subgraph-based methods [8, 41] retrieve query-relevant neighborhoods. These approaches expose different intermediate representations, making optimizations such as path caching, batched traversal, and reuse detection difficult to generalize across systems. Moreover, existing solutions [5, 6, 8, 41] primarily convey retrieved evidence, but lack a common semantic representation with execution metadata such as cross-query reuse, computation cost, memory footprint, and access timing. Consequently, the runtime cannot systematically identify equivalent graph computation across queries or coordinate caching, batching, prefetching, and generation around such reuse.

Proposed Solution: In this task, we aim to develop a common, semantics-preserving intermediate representation (IR) for diverse retrieval algorithms that retains their retrieval semantics while exposing reusable work and execution requirements to the downstream system. Our hypothesis is that an *entity–relation path* provides a useful canonical building block for such an IR. A path preserves the ordered entities and relations that establish provenance and reasoning structure; nodes and triples naturally correspond to zero- and one-hop path units, respectively; and multiple path units can be composed into neighborhoods and subgraphs. Importantly, we do not require every retriever to perform path enumeration. Instead, the path serves as an interchange representation into which the reusable portions of ranking-, traversal-, and subgraph-based retrieval can be mapped, while method-specific scoring and reasoning policies remain unchanged.

The key novelty is to associate each retrieved evidence unit with both *semantic identity* and *execution metadata*. For a path $p = (v_0, r_1, v_1, \ldots, r_\ell, v_\ell)$, we construct a canonical signature from its ordered entity and relation identities, independent of the query that discovered it, its rank, or its position in a prompt. The IR additionally records provenance, relevance scores, composition relationships, and a canonical serialization, together with systems metadata such as token count, estimated prefill cost, KV-cache footprint, and reuse status. Thus, two retrievers that independently discover the same semantic path can expose the same

reusable unit to the runtime. This abstraction does *not* assume that the corresponding KV states are always directly reusable: KV reuse depends on the LLM, prompt prefix, token positions, and execution policy. Instead, the IR preserves sufficient semantic and execution information for the runtime to determine when reuse is legal and profitable. This separation between *retrieval semantics* and *execution policy* is what enables optimizations to transfer across GraphRAG algorithms.

Specifically, we will build the IR around a common hop-constrained path retrieval primitive. Let $G = (V, E, \mathcal{R})$ denote a directed graph and q a query. We represent a GraphRAG retrieval policy using three components: (1) a *query grounding function* $F(q)$ that identifies source–target entity pairs and retrieval constraints; (2) a *path discovery operator* that enumerates valid entity–relation paths within a hop bound k; and (3) a method-specific *selection function* Sel that ranks, filters, or aggregates the discovered paths. For a source–target pair (s, t), we define $\mathcal{P}(s, t, k, G) = \{p \mid p \text{ is a valid } s\text{–}t \text{ path in } G,\ |p| \leq k\}$. The retrieved evidence for query q can then be expressed as $\mathcal{E}_q = \text{Sel}\left(\bigcup_{(s,t)\in F(q)} \mathcal{P}(s, t, k, G)\right)$. The abstraction exposes reuse across queries. When multiple queries contain the same canonical (s, t) pair, the corresponding path set $\mathcal{P}(s, t, k, G)$ needs to be generated only once and can be shared across those queries. More generally, recurring entity–relation paths become stable semantic units in the IR that can be reused by both retrieval and downstream LLM execution.

Task 1.2: Retrieval Algorithm Design for GraphRAG

Beyond unified and semantic-preserving graph abstraction, GraphRAG introduces a new reuse opportunity beyond conventional text-based caching: graph retrieval often produces overlapping paths across queries due to the locality of entities and topics. Such redundancy is inherent to realistic knowledge-intensive workloads, where queries exhibit Zipfian popularity and short-window recurrence [42], and serving systems already exploit shared-context locality through cache-aware scheduling [11, 43]. This opportunity is further amplified in GraphRAG benchmarks, where CWQ is derived from WEB by adding compositional constraints [44], causing related queries to share base entities and relation paths. Our preliminary results indicate 37%–52% redundancy during the prefill stage.

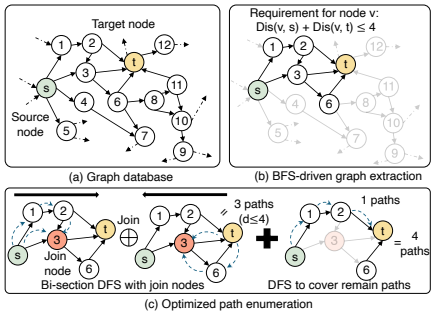


Figure 5: An example of the JbiDFS algorithm.

Proposed Solution: Building on the graph IR in Task 1.1, this task develops an efficient path-discovery algorithm for producing its canonical semantic units while reducing multi-hop retrieval overhead. Efficient hop-constrained path enumeration (HcPE) is a key scalability challenge in GraphRAG retrieval, where the objective is to enumerate all paths of up to k hops between a pair of entities. The search complexity depends strongly on graph connectivity and retrieval depth. Letting $\bar{\Delta} = |E|/|V|$ denote the average out-degree, a depth-bounded exhaustive DFS may explore approximately $O(\bar{\Delta}^k)$ candidate paths. Existing GraphRAG systems reduce retrieval cost through heuristic pruning. For example, Microsoft GraphRAG, LightRAG, HippoRAG, and PathRAG [5, 9, 45, 46] prune communities, ego-networks, personalized PageRank neighborhoods, and low-reliability paths, respectively. While effective for passage- or community-level retrieval, these approaches cannot guarantee preservation of all valid reasoning paths and are therefore unsuitable for enumeration-based GraphRAG. To address this challenge, we propose Join-based Bidirectional DFS (JbiDFS), which preserves the complete path set while substantially reducing the search space through parallel graph traversals, thereby lowering hop-dependent computation and storage overheads.

Stage 1. Pruning via bidirectional BFS. Before enumerating paths, we need to know which vertices could lie on a valid s–t path within the hop budget. A naive approach would run a traversal from s over the entire graph, but the search space grows exponentially with graph size and hop budget. As such, we use parallel BFS, one from s and one from t, to obtain each vertex's shortest-path distance to s and t. If a vertex's distance to s plus its distance to t exceeds the hop bound k, that vertex cannot lie on any s–t path of at most k hops and is discarded; the surviving vertices and edges form a substantially smaller induced subgraph

$G'=(V', E')$. This distance pruning [47] is sound: every discarded vertex is guaranteed absent from any valid path, though G' may still retain a few vertices unused by any simple path. Typically, $|V'|$ is one to two orders of magnitude smaller than $|V|$, reducing the search space that later stages must explore.

Stage 2. Join-vertex selection. The next question is how to break a single k-hop enumeration into smaller sub-problems. Rather than searching for full-length paths in one pass, JbiDFS follows the meet-in-the-middle principle used in bidirectional path enumeration [47]: pick a vertex in G' as a join point, enumerate path segments from s and t separately, and concatenate them at the join. The join point matters because each segment's enumeration cost grows with hop budget, and the total cost of two segments combined is minimized only when their hop budgets are close to each other. JbiDFS selects the join vertex j as the midpoint of a shortest s–t path; when the shortest-path length is close to k, this naturally balances the two segments' hop budgets.

Stage 3. Bounded DFS and join. With j fixed, the remaining task is to enumerate the two path segments and join them into complete s-to-t paths. We perform a hop-bounded forward DFS from s to j and a hop-bounded reverse DFS from t to j, both restricted to the pruned subgraph G'. The former enumerates all feasible $s \rightarrow j$ prefixes, while the latter enumerates feasible $j \rightarrow t$ suffixes after reversing the discovered paths. We then join each prefix–suffix pair at j, retaining only those whose combined hop count does not exceed k and whose vertex sets intersect only at j, thereby ensuring that the resulting path is simple. This procedure enumerates all feasible s-to-t paths through j contained in G'. Because both searches operate on G', whose pruning substantially reduces the effective branching factor relative to the original graph G, both the DFS search space and the number of prefix–suffix combinations are significantly reduced. Consequently, when G' is substantially sparser than G, the enumeration cost is considerably lower than that of an exhaustive k-hop DFS.

Stage 4. Covering residual paths. Not all valid paths pass through the join vertex j, so bounded DFS alone is incomplete. To recover the remaining paths, we remove j from G' and perform an additional hop-bounded DFS to enumerate all valid s-to-t paths that avoid j. Since the paths through j and those avoiding j form a disjoint partition of the valid path set, completeness is guaranteed. In Fig. 5, bidirectional BFS yields $\mathcal{V}$=1,2,3,6 and selects node 3 as the join vertex. Joining two prefixes and two suffixes produces three complete paths, while the residual DFS recovers the remaining path $s \rightarrow 1 \rightarrow 2 \rightarrow t$.

Risk Management: All retrieval algorithms may introduce a small accuracy variation (typically within 1–2%), depending on the retrieval strategy, query characteristics, and graph structure. For example, imposing a hop bound may improve accuracy on dense CWQ graphs by excluding noisy long-tail paths, while slightly reducing accuracy on sparser MetaQA graphs when useful evidence lies along paths longer than the selected hop bound. To mitigate such cases, we will incorporate a lightweight semantic-consistency check between the query and the preliminary answer, using embedding-based similarity or Natural Language Inference-style entailment. This mechanism will be activated when a retrieval optimization causes an undesirable accuracy degradation for specific workloads or use cases.

3.2. Research Thrust 2: Retrieval-Context-Aware Memory Optimization

Thrust 1 exposes reusable semantic structure in GraphRAG retrieval; Thrust 2 uses this information as lookahead for memory optimization. Modern GraphRAG pipelines span CPU graph processing, GPU inference, high-bandwidth memory (HBM), host memory, and increasingly CXL [48] or other extended-memory tiers. These components differ substantially in capacity, bandwidth, latency, and computational capability, making resource placement and data movement critical to performance. We use *Mixture-of-Experts (MoE) LLM* [49] serving as a representative and demanding model-side setting because the large expert parameter footprint creates substantial demands for heterogeneous placement, expert caching, and inter-tier data movement. Existing systems largely react to resource demand using heuristics, profiling-based metrics, or model-internal execution signals, without exploiting information exposed during GraphRAG retrieval, e.g., entities, reasoning paths, context structure, and cross-query reuse. Thus, we investigate how this semantic lookahead from retrieval can guide three complementary memory decisions: Task 2.1, context-aware MoE expert placement and fidelity allocation; Task 2.2, retrieval-guided expert-cache management and prefetch-

ing; and Task 2.3, semantic-path-based KV-cache reuse and management.

Task 2.1: Retrieval- and Context-Guided Resource Orchestration

Existing LLM serving systems adapt resource allocation via runtime precision adaptation [50–54] and heterogeneous CPU–GPU or near-memory execution [55–59]. However, these mechanisms typically make resource placement and fidelity decisions from cache state, global workload statistics, short-horizon routing signals, or model-internal execution behavior. In conventional GPU-offloading systems, such reactive or locally predicted decisions can lead to frequent expert-cache misses and repeated transfers of large expert weights [52, 53, 60–63]; GPU–NDP systems reduce host–GPU weight movement by executing non-resident experts near memory, but static placement can mismatch dynamic demand, while reactive migration still incurs repeated cross-device and NDP-internal data movement [64–69]. Task 2.1 therefore investigates how retrieval-derived context and early execution behavior can be combined into a request-specific utility signal for proactive heterogeneous placement and fidelity allocation.

Preliminary Result: We study whether complementary prefill signals improve context-specific expert-importance estimation. Fig. 6 evaluates combinations of expert activation frequency and routing confidence on Mixtral-8×7B [32]. The balanced setting $\alpha = 0.5$ achieves the highest average accuracy of 66.90%, improving over the two single-signal settings by 0.69 and 1.96 percentage points, while throughput varies by less than 0.6%. These results show that combining complementary early-execution signals provides more effective resource-importance estimation with negligible overhead, motivating their integration with retrieval-derived context in the proposed resource-utility model.

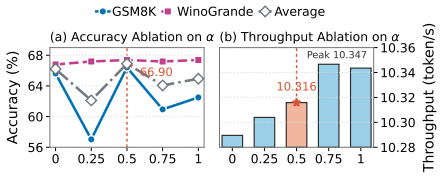


Figure 6: Accuracy–throughput ablation on Mixtral-8×7B.

Proposed Solution: We will develop a context-aware orchestration framework that jointly optimizes data quantization and placement across heterogeneous memory hierarchies and technologies. In the proposed model, we will study how an MoE expert weight block can be placed across heterogeneous memory and execution tiers, transferred on demand, and represented at different numerical precisions. Using retrieval and early-execution signals, the framework jointly determines: (1) the importance of each expert to the current context, (2) which experts should remain resident in the fast tier, and (3) the numerical precision of experts outside the fast tier under memory-capacity and bandwidth constraints.

Context-Aware Resource Utility Estimation. For each expert r, we define utility U_r as its expected importance to the upcoming generation under the current retrieved context. Before prefill, we derive a retrieval prior R_r that estimates how strongly the retrieved entities and reasoning paths are expected to invoke expert r, using path relevance and frequency, reasoning depth, entity/relation composition, and cross-query reuse. During prefill, this prior is refined using the normalized activation frequency P_r of expert r and its routing confidence W_r as $U_r = \beta\widetilde{R}_r + (1-\beta)\left[\alpha\widetilde{P}_r + (1-\alpha)\widetilde{W}_r\right]$, where β balances retrieval-derived lookahead and observed execution behavior, while α balances how often an expert is selected and how strongly it is selected. A larger U_r therefore indicates that expert r is more likely to contribute substantially to the upcoming generation and should receive higher system priority. We will characterize how quickly this ranking stabilizes from retrieval through prefill, enabling early commitment when the top-K expert set is reliable and lightweight refinement otherwise. This will further provide the theoretical foundation for expert-level numerical precision selection and data locality optimization.

Context-Aware Heterogeneous Resource Placement. As modern AI systems increasingly span CPU/GPU caches and heterogeneous external memory, effective data placement becomes critical to reducing access latency and data movement. The utility ranking determines which expert weights should remain resident in the fast execution tier, such as cache and HBM, and which can remain in slower tiers such as host or CXL-attached memory or near-memory devices. Let m_r denote the memory footprint of expert r, and let B_r denote the transfer or execution stall avoided when that expert is locally resident. We select the fast-tier resident expert set $\mathcal{S}$ as $\arg\max_{\mathcal{S}} \sum_{r\in\mathcal{S}} U_r B_r$, subject to $\sum_{r\in\mathcal{S}} m_r \le M_{\text{fast}}$.

Thus, an expert receives high residency priority when it is both important to the current context and expensive to access remotely. Rather than migrating experts after individual routing events, the runtime forms

a coarse-grained, context-stable placement plan for the upcoming generation. For concurrent requests, expert utilities are aggregated across queries so that experts expected to serve multiple semantically related requests receive higher fast-tier residency priority.

Adaptive Numerical Precision for Expert Quantization. Quantization has been widely adopted in MoE models to reduce expert weight storage and computation overhead. Experts that remain outside the fast tier are assigned context-dependent fidelity according to their utility and approximation sensitivity. Offline, each expert r is profiled under fidelity levels $b \in \mathcal{B}$, such as different numerical precisions or compressed representations, to obtain sensitivity $\Delta_{r,b}$. At runtime, we select the representation of each nonresident expert as $\min_{b_r} \sum_{r \notin \mathcal{R}\text{fast}} U_r \Delta r, b_r$, subject to $\sum_{r \notin \mathcal{R}_{\text{fast}}} C(b_r) \leq B_{\text{resource}}$, where $C(b)$ captures the storage or transfer cost of fidelity level b. High-utility or approximation-sensitive experts retain higher precision, while experts predicted to contribute less to the current context are represented more aggressively at lower fidelity. Precomputed sensitivity profiles and lightweight ranked selection keep runtime overhead low.

Task 2.2: Retrieval-Guided Expert Cache Management

Even with placement and quantization optimizations, local cache capacity remains limited, requiring the runtime to decide which experts should remain resident, which should be evicted, and which should be prefetched. Existing MoE serving systems reduce expert-related memory and execution overhead through activation-aware caching [70, 71] and expert prefetching [56, 61], while general LLM cache management relies on execution history or model-internal signals such as prefix reuse, recency, or runtime importance [10, 11, 20, 72]. This task investigates whether retrieval-derived structural identity can provide earlier and more stable prediction of future expert demand, enabling proactive cache admission, replacement, residency, and prefetching.

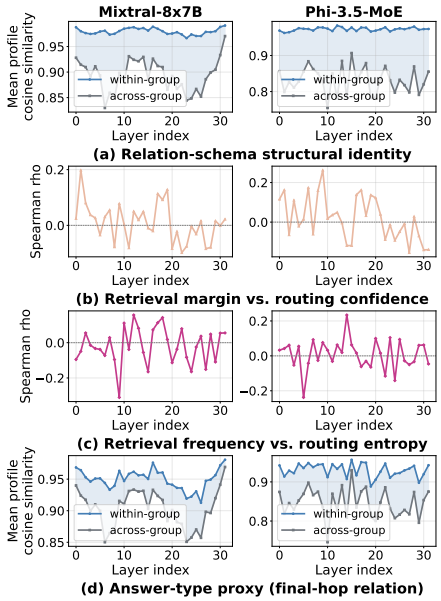


Figure 7: Comparison over 4 regimes across 2 models on RoG-CWQ datasets.

Preliminary Result: We study which retrieval-derived signals best predict downstream expert-routing behavior. Fig. 7 compares four candidate signals against layer-wise expert-activation profiles on Mixtral-8×7B [32] and Phi-3.5-MoE [33]. Relation-schema identity (Fig. 7a) achieves within-group profile similarity above 0.96 across all layers for both models and maintains a stable margin over the across-group baseline. The coarser final-hop relation signal (Fig. 7d) follows the same trend but with lower and less stable separation. In contrast, retrieval margin and retrieval frequency (Fig. 7b–c) show no consistent correlation with routing behavior. These results indicate that retrieval-derived structural identity provides a stable and architecture-consistent early predictor of expert demand, motivating its use to construct $\widetilde{R}_r$ in the proposed resource-utility model for proactive expert-cache management.

Proposed Solution: Guided by the preliminary result, we instantiate the retrieval-derived signal $\widetilde{R}_r$ from Task 2.1 using structural identity rather than continuous embedding similarity. We use this signal to guide expert-cache admission, residency, replacement, and prefetching before prefill begins. Specifically, for a retrieved path p, we define its relation-type schema as the ordered sequence of relation types, independent of the specific entities, $\sigma(p) = (\text{rel}_1, \text{rel}_2, \ldots, \text{rel}_k)$. Our preliminary results show that paths sharing the same schema induce similar expert-activation profiles across layers. We therefore use $\sigma(p)$ as a cache key and reuse the expert-demand profile previously observed for that schema. Specifically, if $\sigma(p) \in \mathcal{C}$, where $\mathcal{C}$ denotes the set of schemas with sufficient prior support, we obtain the estimated expert-demand profile from the cache as $\widetilde{R}_r \leftarrow (\widetilde{P}_r, \widetilde{W}r)\text{cached}[\sigma(p)]$; otherwise, we derive the profile using $\text{fallback}(p)$.

The predicted profile provides semantic lookahead for expert-cache management. Experts consistently associated with the retrieved schemas receive higher cache admission and residency priority and can be prefetched before prefill, while experts with low predicted demand are preferentially evicted when cache capacity is constrained. For unseen or insufficiently observed schemas, the fallback is constructed from

weaker signals such as the final-hop relation type or retrieval embedding. We reflect prediction confidence in the utility model of Task 2.1 by allowing $\beta = g(|\mathcal{C}(\sigma(p))|)$, where g increases with schema support. Thus, well-supported structural matches receive greater weight, while novel or uncertain schemas rely more heavily on execution-derived signals. Because schema matching requires only lightweight structural lookup, this mechanism provides early expert-demand prediction with minimal retrieval-side overhead. We will further investigate how these insights can guide the redesign of cache, memory hardware, and replacement mechanisms to enable retrieval-informed data management policies. Furthermore, replacement and prefetching policies should be co-designed to optimize data locality, since aggressive prefetching can evict useful cache contents and degrade locality.

Task 2.3: Context-Aware KV Cache Management

Modern LLM serving uses KV caching to avoid recomputing prompt states during autoregressive decoding. Existing systems extend KV reuse across requests through identical prefixes [10, 11], reusable prompt modules [22, 73], and position-independent or selectively recomputed text chunks [36–39]. However, GraphRAG exposes a different form of reuse: the same entity–relation reasoning path can recur across queries despite appearing at different prompt positions or within different surrounding contexts. Another challenge is deciding which reusable paths should occupy the finite shared KV cache. Existing GPU KV-cache management largely treats reuse as an online problem because future requests are unknown: systems such as vLLM and SGLang [10, 11] manage cache state online, while ALISA [20] predicts future importance from runtime attention patterns. More generally, cache-replacement policies such as Hawkeye [72] estimate future accesses from execution history.

Preliminary Results: We study how much cross-query path reuse in GraphRAG is missed by reactive caching [10, 11], and whether short-horizon scheduling [20] can recover it. Fig. 8(a) evaluates six replacement policies on trace-driven KV-path accesses from two KGQA benchmarks (RoG-CWQ, MetaQA-3hop), across cache capacities normalized to each workload's distinct reusable paths. At 10% capacity, LRU, which is used by today's prefix-caching systems, achieves only 18.7% hit rate on CWQ, versus 37.8% for Belady's offline replacement policy [72] and 41.1% for a static retrieval-informed frequency policy; LFU reaches 25.6%. The Oracle Ceiling is a fixed, order-independent policy that permanently caches the globally most frequent paths using full-trace frequency information, and thus is not a true optimal bound. On MQA, reactive policies converge to ∼1.1% hit rate, reflecting its much lower path redundancy. Fig. 8(b) isolates scheduling: reordering CWQ queries within a bounded lookahead window does not improve LRU, falling to 14.2% at a 256-query window versus 18.7% without reordering, and reaches 38.2% only with full-workload visibility ($N = 3{,}531$), still 3 points below the static ceiling. Thus, the missing reuse is primarily an admission problem, not an eviction or short-horizon scheduling problem: effective reuse requires retrieval-exposed knowledge of which paths are likely to recur, motivating our retrieval-informed admission and placement policy for the shared KV pool.

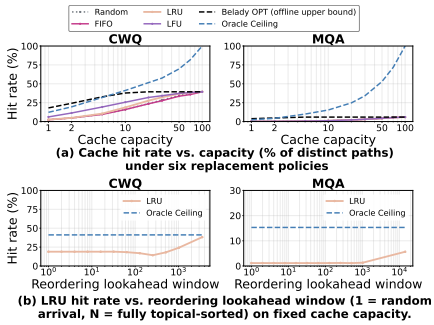


Figure 8: Comparison across multiple cache replacement policies on RoG-CWQ and MetaQA-3hop datasets.

Proposed Solution: Transformer-based LLM inference maintains a KV cache that stores previously computed key–value tensors, avoiding repeated projection of prior tokens during autoregressive decoding. Let $x_{1:n}$ denote the prompt, L the number of transformer layers, d the hidden dimension, and $K_\ell, V_\ell \in \mathbb{R}^{n \times d}$ the key and value tensors at layer ℓ. Without KV caching, each decoding step must recompute the key and value states of preceding tokens; with caching, previously computed states are retained and only those of newly generated tokens are appended. Existing KV reuse techniques, including prefix caching [10, 11, 22, 73] and chunk-level reuse [36–39], primarily exploit repeated prefixes or reusable text chunks. These assumptions do not naturally fit GraphRAG, where the same semantic reasoning path can recur across queries at different positions and within different surrounding contexts.

Semantic Path KV Cache Reuse. We will couple the reusable graph-path abstraction with a GPU-resident

KV representation that can be computed once and reused across queries. For each retrieved path π, we define its *semantic path KV unit*, denoted by $\Phi(\pi)$, as the collection of per-layer key and value tensors associated with the tokens representing π. For LLMs using position-dependent transformations such as Rotary Positional Embedding (RoPE) [74], keys are stored in a position-independent form and realigned to the target sequence positions when reused. The pair $(\pi, \Phi(\pi))$ therefore becomes the fundamental unit for KV admission, placement, reuse, and eviction. Unlike prefix- or chunk-based caching, this abstraction enables reuse based on graph semantics even when the same path appears at different positions across prompts. Our preliminary results show a 2.1×–3.5× throughput improvement across configurations at moderate batch sizes. Beyond prefill reuse, the retrieved path representation can also guide decoding. The ordered entity–relation sequence provides a lightweight prior for speculative decoding [75], while path-level signals such as branching factor and relevance-score entropy can guide verification effort and decoding precision. We will also evaluate path-based reuse across transformer layers, as context sensitivity may vary by layer.

Reuse-Aware Query Ordering. A shared KV cache eliminates redundant KV computation, but arbitrary query ordering can still cause repeated data movement and premature eviction of shared states. GraphRAG exposes retrieved reasoning paths before generation, allowing cross-query KV reuse to be predicted in advance. We will therefore schedule queries to maximize path overlap between consecutively executed requests, increasing the likelihood that reusable KV states remain resident in fast GPU memory and caches. At each scheduling step, the next query is selected according to its expected KV reuse with recently executed queries, and the reuse state is updated after execution. The same path-overlap signal can also guide expert-cache reuse. Since relation-schema identity predicts expert activation, path-overlap ordering can jointly improve KV- and expert-cache reuse. Building on session-level locality in Fig. 2(b), we will explore predictive clustering that anticipates follow-up queries and reserves cache capacity accordingly. In multi-tenant settings, scheduling will jointly optimize request routing and execution order to maximize reuse within shared cache partitions.

3.3. Research Thrust 3: Context-Aware System and Hardware Optimization

Thrust 2 uses retrieval-derived context to guide resource placement, cache management, and numerical-precision decisions; Thrust 3 develops the system- and hardware-level mechanisms needed to efficiently realize these optimizations. This is particularly challenging in GraphRAG, where irregular graph processing and dynamic model demand span CPUs, GPUs, and heterogeneous memory hierarchies. Existing runtime mechanisms primarily rely on model-side signals, such as routing behavior, resource availability, and execution dependencies, while largely overlooking retrieval-derived semantics. In this thrust, we will develop context-aware system and hardware mechanisms that jointly leverage retrieval context and runtime state to optimize data movement, scheduling, and execution for both model inference and graph retrieval.

Task 3.1: Context-Aware Data Movement and Execution Scheduling

In MoE LLMs, routed experts differ substantially in model contribution, transfer cost, and access latency. Existing methods adapt expert execution through pruning [71, 76, 77] and communication allocation based on offline expert profiles [78–80] or current router outputs [52, 81]. MoE serving systems further reduce expert-transfer overhead through caching and route prediction [60, 70, 78, 82], prefetching and pipelining [61, 62, 83, 84], and adaptive expert execution [85–87]. However, these approaches primarily rely on offline profiles or model-side runtime signals and optimize allocation and scheduling largely in isolation. They therefore do not jointly determine *which* nonresident experts should consume limited bandwidth, *how* selected transfers should be organized, and *when* they should occur relative to computation.

Preliminary Result: Our preliminary study shows that uniformly allocating a limited communication and execution budget across activated experts is inefficient. Fig. 9 shows that selectively prioritizing high-importance experts consistently improves the accuracy–bandwidth Pareto frontier. For INT2 DeepSeekMoE-16B, selective allocation improves MMLU from 29.98 to 37.72 and exceeds the best uniform configuration by 3.44 percentage points at comparable transfer overhead, while reducing decoding latency by up to 1.44×. This motivates utility-guided allocation of limited communication and execution resources.

Proposed Solution: We will develop three coordinated techniques: budget-aware selective transfer, context-aware transfer coalescing, and communication–computation overlap.

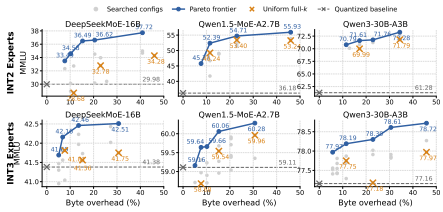


Figure 9: MMLU–overhead Pareto frontier for INT2/INT3 MoE experts, showing the advantage of selective over uniform compensation.

Budget-Aware Selective Transfer. The runtime uses the utility estimates from Task 2.1 to determine which nonresident experts should receive preferential transfer or enhanced execution. Let $\mathcal{A}$ denote the experts required by the current execution phase, U_r the utility of expert r, and C_r its transfer or execution cost. Given budget B, we select the optimal set $\mathcal{C}$ as $\mathcal{C} = \arg\max_{\mathcal{C}\subseteq\mathcal{A}} \sum_{r\in\mathcal{C}} U_r$, subject to $\sum_{r\in\mathcal{C}} C_r \leq B$. The budget adapts to available bandwidth, memory pressure, and latency slack, while the constrained allocation prevents expensive low-benefit experts from dominating limited communication capacity.

Context-Aware Transfer Coalescing. Selected experts will be grouped by destination, execution phase, and semantic or structural locality to reduce fragmented data movement. Retrieved paths and query clusters provide natural grouping signals, allowing experts associated with overlapping entities, paths, or related requests to be laid out and transferred in larger contiguous regions. Transfer groups will be formed subject to size and timing constraints to balance bandwidth efficiency against execution urgency.

Communication–Computation Overlap. We will overlap selected transfers with independent computation using asynchronous execution and transfer queues. For each expert r, we estimate transfer time T_r^{mov} and available computation slack T_r^{slack}. Transfers satisfying $T_r^{\text{mov}} \leq T_r^{\text{slack}}$ can be hidden, while exposed transfers are prioritized according to expected stall reduction. For concurrent Graph-based RAG requests, the runtime further exploits cross-query reuse to avoid redundant transfers. These techniques convert context-aware resource decisions into end-to-end system efficiency.

Task 3.2: Hardware Support for In-Memory Retrieval Acceleration

Existing graph accelerators [88–95] are largely optimized for fixed traversal models or predictable access patterns, limiting support for dynamic, path-centric retrieval. Vertex-centric designs [88–92] efficiently support regular propagation but lack native support for path enumeration, runtime backtracking, and dynamic branching. Subgraph-mining accelerators [94] offer richer enumeration support, but do not target the retrieval primitives in Task 1.2. Thus, existing hardware is not well suited to the dynamic retrieval required by our GraphRAG framework.

Proposed Solution: The goal of our accelerator is to enable (1) unified hardware support for the two structurally distinct phases of JbiDFS – a regular, frontier-based pruning phase and an irregular, backtracking-heavy path enumeration phase – without resorting to separately customized pipelines for each, and (2) full exploitation of both path-level and segment-level (prefix/suffix) parallelism without incurring the overhead of an explicitly software-managed backtracking stack. The proposed accelerator includes (i) a near-memory bidirectional pruning tile array, (ii) a lightweight join-vertex selection stage, and (iii) a speculative path enumeration engine. An overview of the proposed architecture is depicted in Fig. 10.

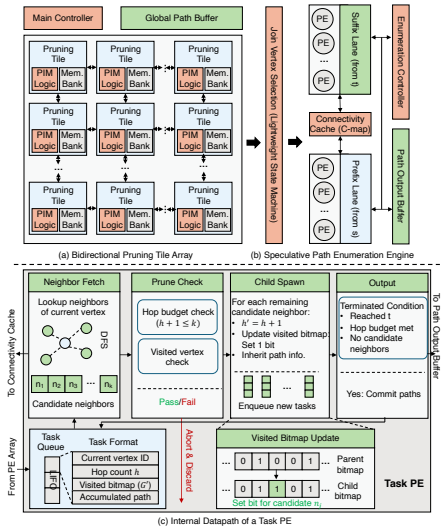


Figure 10: The overview of proposed hardware.

Bidirectional Pruning Tile Array. As shown in Fig. 10 (a), the main controller dispatches each query's search boundaries to a mesh of near-memory pruning tiles, with intermediate frontier state held in a global path buffer. Each tile offloads the per-round frontier-expansion updates (distance labels and visited markers of both search directions) to memory-side logic, avoiding repeated round-trip movement of frontier data across the memory hierarchy. Join-vertex selection requires only comparing the distance labels already

computed by the pruning tile array from both search directions and can therefore be implemented using a lightweight state machine.

Speculative Path Enumeration Engine. Hop-constrained path enumeration with backtracking is the component of JbiDFS that existing accelerators fail to cover. Pattern-aware graph mining accelerators such as FlexMiner [94] generate a compiler-produced execution plan and an on-chip connectivity map sized for a statically known pattern, and therefore cannot accommodate a hop budget or a bidirectional pruning boundary that is only determined at query time. Pipeline architectures that decouple regular from irregular workloads, such as Fifer and Phloem [96, 97], improve scheduling flexibility, but this decoupling operates at task granularity across independent workloads and never exposes backtracking itself as a hardware-level primitive. As shown in Fig. 10(b), the enumeration engine is organized as two independently issuing lanes of task processing elements (PEs) – a prefix lane rooted at s and a suffix lane rooted at t – that share a single on-chip connectivity cache (C-map), so that adjacency information for vertices near the join point is fetched once and reused across both lanes.

Fig. 10(c) details the internal datapath of each task PE: a task queue holds pending branch-expansion tasks; the neighbor-fetch stage looks up candidate neighbors through the shared C-map; the prune-check stage evaluates the hop budget and visited-set constraint for each candidate; and it speculatively issues one new task per surviving candidate back into the task queue. The accelerator will be prototyped on FPGAs.

3.4. Research Thrust 4: Benchmarking, Simulation, and Application Test Cases

GraphRAG systems are attracting increasing research interest, yet the community lacks unified workload characterization, reusable simulation infrastructure, and benchmark suites for evaluating cross-layer interactions among graph retrieval, model execution, memory systems, and heterogeneous resources. We will develop a cross-layer benchmarking and simulation framework that connects workload characteristics to retrieval behavior, memory traffic, and end-to-end system performance. The framework will support controlled workload characterization, calibrated architectural simulation, and two representative application test cases in EDA hardware code generation and biomedical knowledge reasoning.

Task 4.1: Graph-based RAG Benchmark Development

We will establish a benchmark suite that captures the diverse structural and runtime characteristics of Graph-based RAG workloads, including ComplexWebQuestions (CWQ), WebQSP, and MetaQA. The benchmark will report both task-level and system-level metrics, including retrieval quality, retrieval latency, prefill and decoding latency, cache hit rate, memory footprint, host–accelerator traffic, bandwidth utilization, throughput, and energy.

Task 4.2: Cross-Layer Simulation and System Prototyping

We will develop a multi-level evaluation infrastructure that combines real-system execution with configurable architectural simulation. The proposed graph retrieval, context-management, resource-orchestration, and scheduling techniques will be implemented using Python and PyTorch and evaluated on commercial CPU–GPU platforms. Fine-grained traces of computation, memory accesses, transfers, cache behavior, and synchronization stalls will drive a simulator modeling heterogeneous compute, accelerator and host memory, cache hierarchies, interconnects, and alternative memory tiers. The simulator will be calibrated and validated against measurements from real GPU systems and then used to explore configurations not directly available on commodity hardware. Sensitivity studies will quantify the effects of memory capacity, interconnect bandwidth, workload concurrency, context locality, and resource-placement policies on Graph-based RAG performance and energy efficiency.

Task 4.3: End-to-End Application Test Cases

We will evaluate the integrated framework on two application domains with substantially different graph structures and reasoning requirements.

Test Case 1: Graph-based RAG for Hardware Code Generation. We will construct a hardware-design knowledge graph linking specifications, HLS kernels, optimization directives, RTL modules, synthesis results, and verification artifacts [98–100]. Graph-based RAG will support HLS/RTL generation and optimiza-

tion, evaluated using functional correctness, synthesis success, latency, initiation interval, FPGA resource utilization, and end-to-end system metrics.

Test Case 2: Biomedical Knowledge Reasoning. We will evaluate the framework on heterogeneous biomedical knowledge graphs such as PrimeKG [101] and Hetionet [102], which integrate relationships among diseases, drugs, genes/proteins, pathways, and other biomedical entities. We will construct multi-hop and evidence-grounded reasoning workloads using biomedical QA resources such as BioASQ [103, 104]. Evaluation will consider answer quality, evidence/path recall, and system metrics including retrieval latency, KV-cache footprint, memory traffic, throughput, and end-to-end latency.

4. Evaluation, Integration, and Project Management

4.1. Evaluation Plan

To assess the proposed framework, we will evaluate techniques at the retrieval, memory-management, runtime, hardware, and end-to-end GraphRAG levels using the benchmarking, simulation, and application-validation infrastructure developed in Research Thrust 4. Retrieval, context-management, resource-orchestration, and runtime algorithms will be implemented in Python and PyTorch and evaluated on commercial GPUs and heterogeneous CPU–GPU platforms, while hardware mechanisms will be studied using the simulation and prototyping infrastructure in Task 4.2. Each thrust will be evaluated independently and within the integrated system across varying graph structures, retrieval depths, context lengths, model configurations, workload concurrency, and resource constraints.

For Thrust 1, we will evaluate the semantic-preserving IR and JbiDFS using semantic fidelity, retrieval recall and completeness, path coverage, cross-query reuse, search-space reduction, retrieval latency, and IR-construction overhead. For Thrust 2, we will evaluate retrieval-context-aware memory optimization using expert-demand prediction accuracy, utility-ranking stability, placement and precision decisions, expert- and KV-cache hit rates, prefetch accuracy, KV-cache footprint, memory utilization, and downstream accuracy. For Thrust 3, we will evaluate runtime and hardware optimization using data-transfer volume, bandwidth utilization, communication stalls, communication–computation overlap, time-to-first-token, inter-token latency, throughput, end-to-end latency, graph-retrieval throughput, memory traffic, execution time, and energy efficiency. We will compare against representative graph-retrieval, MoE-serving, expert-cache, KV-cache, heterogeneous-inference, and graph-acceleration baselines under matched model, memory, bandwidth, and quality constraints. Controlled ablations will isolate the effects of retrieval-derived context, early execution signals, semantic reuse, placement, precision, cache prediction, semantic-path KV reuse, selective data movement, transfer coalescing, scheduling, and architectural support. For quality–efficiency tradeoffs, we will report Pareto frontiers under matched resource budgets. Finally, we will evaluate the integrated framework using the HLS/RTL code-generation and biomedical knowledge-reasoning test cases in Thrust 4 and study scalability with graph size, retrieval depth, context length, model size, batch size, request concurrency, and inter-query locality. Overall success will be measured by improvements in retrieval efficiency, memory footprint, communication volume, latency, throughput, and energy efficiency while preserving retrieval quality and downstream accuracy.

4.2. Timeline and Milestones

Table 1 shows the planned research and education activities. The four research thrusts, evaluation, and education plans will be executed collaboratively by the PIs' groups, with complementary expertise in graph and machine learning, AI systems, computer architecture, and memory systems. The research will be considered successful if the following milestones are achieved: 1) development of unified GraphRAG representations and efficient retrieval algorithms that preserve semantic information and re-

Table 1: Timeline of proposed research and education activities

Tasks	Year 1	Year 2	Year 3	Year 4
T1: Unified Graph Abstractions and Retrieval Algorithms	✓	✓	✓	
T2: Retrieval-Context-Aware Memory Optimization		✓	✓	✓
T3: Context-Aware System and Hardware Optimization		✓	✓	✓
T4: Benchmarking, Simulation, and Application Validation	✓	✓	✓	✓
Courses and Training	✓	✓	✓	✓

duce multi-hop retrieval overhead while maintaining retrieval completeness and quality; 2) development of retrieval-context-aware memory-optimization techniques that improve expert placement, numerical precision, expert-cache management, and semantic-path KV-cache reuse; 3) development of context-aware system and hardware mechanisms that reduce data movement and communication overhead through selective transfer, transfer coalescing, communication–computation overlap, and near-memory graph-retrieval acceleration; and 4) integration of the proposed techniques into an end-to-end GraphRAG system and comprehensive evaluation across representative datasets, models, and hardware platforms to demonstrate scalability, efficiency, and retrieval and generation quality. The outcomes will provide algorithmic, system, memory-management, and architectural advances for scalable GraphRAG and establish an integrated framework for studying emerging retrieval-augmented AI workloads across the computing stack.

5. Intellectual Merit

The proposed research will advance the foundations of retrieval-augmented AI systems by introducing **retrieval-informed cross-layer system design** that integrates graph retrieval, LLM execution, runtime optimization, and memory systems. By moving beyond retrieval-agnostic system optimization, the project will establish new abstractions and mechanisms that preserve and exploit retrieval-derived semantic and structural information to guide computation, communication, KV-cache management, and resource allocation across the computing stack. A key intellectual contribution is the development of unified GraphRAG representations, efficient retrieval algorithms, and context-aware system mechanisms that exploit semantic locality, evidence importance, and cross-query reuse to reduce redundant computation and data movement. The project will also create benchmarking and simulation infrastructure for systematically evaluating interactions among retrieval, generation, runtime, and architecture. Through this integration of algorithms, systems, and architecture, the project will generate new insights into how retrieval-derived context can serve as a first-class systems signal for scalable and efficient retrieval-augmented and agentic AI.

6. Outreach and Workforce Development

Graduate and Undergraduate Research Training: The project will support graduate students across the three institutions and engage undergraduate researchers through independent studies, senior projects, honors theses, and potential NSF REU supplements. Graduate students will receive cross-layer training in graph retrieval, LLM systems, runtime optimization, memory systems, and computer architecture, and will collaborate across institutions on common software, benchmarks, and system prototypes. Undergraduate students will initially participate in benchmarking, workload characterization, profiling, and software development before progressing to more advanced research tasks. Students will also be encouraged to pursue internships and engage with industry and national-laboratory collaborators, connecting project research with emerging real-world AI systems.

Curriculum Development: Research outcomes will be incorporated into undergraduate and graduate courses at UCF, GWU, and NDSU covering computer architecture, machine learning, AI systems, and memory systems. At UCF, PI Zheng will integrate GraphRAG, LLM serving, graph processing, and hardware–software co-design into *Computer Architecture Design for AI/ML*. At GWU, PI Wu will incorporate graph-based retrieval and context-aware LLM system optimization into *ECE 4810/6210 Machine Intelligence*. At NDSU, PI George will integrate memory and cache optimization techniques into courses related to *ECE 474/674 Computer Architecture*, *ECE 423/623 VLSI Design* and *ECE 726 – Advanced VLSI Design*.

Student Competitions, Tutorials, and Workshops: Building on PI Wu's organization of the DAC University Demonstration, ICCAD CADathlon, DAC 2026 tutorial *"From Vibe Coding to Silicon: Engineering Agentic Systems for Hardware Design"*, and DAC 2026 Early Career Workshop, the PIs will continue developing student competitions, demonstrations, tutorials, workshops, and professional-development activities at major EDA and architecture conferences. These activities will connect students and early-career researchers with emerging research in AI systems, computer architecture, and AI-assisted hardware design, while promoting interaction with academic and industrial communities.

K–12 Outreach: The PIs will develop accessible, interactive demonstrations showing how generative AI

systems retrieve and reason over structured knowledge. Students will construct small knowledge graphs, explore relationships among connected entities, and compare responses generated with and without retrieved information, providing intuitive introductions to graphs, search, AI, and computing systems. These materials will be used in UCF STEM outreach activities and adapted for local outreach at GWU and NDSU.

Education Success: The success of the educational activities will be assessed through student feedback in courses, participation in research and outreach activities, and research outcomes produced by graduate and undergraduate students, including publications, open-source software, research presentations, and student career placement. The PIs will regularly review these outcomes during meetings and use student and institutional feedback to identify opportunities for improving the educational activities and training.

7. Broader Impacts

Broader Research Impact: Successful completion of this project will benefit: 1) *artificial intelligence*, by enabling scalable Graph-based RAG systems that incorporate structured external knowledge into generative AI applications; 2) *computer systems*, by developing cross-layer optimization techniques spanning retrieval, context construction, LLM inference, scheduling, and data movement; and 3) *computer architecture*, by creating context-aware memory and hardware mechanisms for irregular and data-intensive Graph-based RAG workloads. The proposed algorithms, runtime mechanisms, and architectural support will reduce redundant computation, memory use, and data movement, lowering the computational and energy costs of knowledge-intensive AI applications. Open benchmarking and simulation infrastructure will further support reproducible research across the algorithm, system, and architecture layers.

Educational Impact: The project will provide interdisciplinary training for three graduate students and several undergraduate researchers across UCF, GWU, and NDSU, with additional undergraduate participation pursued through REU supplements. The three institutions provide strong environments for recruiting and mentoring students in computer science and computer engineering. Students will receive training in graph algorithms, AI systems, computer architecture, memory systems, and hardware–software co-design, while research outcomes will be incorporated into university courses and K–12 STEM outreach activities.

Communication and Dissemination: To ensure timely dissemination of the project's results, the PIs will publish research findings in leading conferences and journals spanning computer architecture, computer systems, and artificial intelligence, including venues such as ISCA, MICRO, HPCA, ASPLOS, MLSys, SC, and related IEEE/ACM journals. Research findings will also be disseminated through invited seminars, tutorials, workshops, and interactions with semiconductor, AI-system, and cloud-computing companies. The PIs will develop and release open-source implementations of the proposed GraphRAG algorithms, runtime optimizations, and profiling and simulation infrastructures whenever possible. A dedicated project website will be created and maintained to host software, GraphRAG benchmarks and datasets, research publications, educational materials, and documentation, as detailed in the Data Management Plan.

8. Results from Prior NSF Support

PI Wu has no prior NSF support.

PI Zheng is supported by NSF #2441973: "CAREER: A Scalable, Polymorphic, and Efficient Architecture for Irregular and Sparse Computations (APEX)", ($550,000, 3/1/2025-2/28/2030). **Intellectual Merit:** The project will design domain-specific accelerators for irregular and complex computations ranging from fundamental theory to computer architecture. Results of this project have been published [98, 105–111]. **Broader Impacts:** The project will advance the theoretical understanding of irregular and complex applications. **No Overlap:** This project has no overlap with this NSF effort.

PI George was supported by NSF #2246149: "CRII: FET: Ferroelectric FET based flexible circuits and system design", *02/2023 to 01/2026,$174,999*. **Intellectual Merit:** Develop FeFET-based peripherals for memories and effective data caching schemes. **Broader Impact:** Development of peripheral-based computing and security systems; involvement of underrepresented groups and undergraduate students. **Publications**: 4 published papers [112–115], one provisional patent filed, one patent under submission, 2 papers under submission. **No Overlap:** This project has no overlap with this NSF effort.

References Cited

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2023.

[2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*, volume 36, pages 68539–68551, 2023.

[3] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of ACM Symposium on User Interface Software and Technology (UIST)*, pages 1–22. Association for Computing Machinery, 2023.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[6] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2024.

[7] Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. Knowledge graph prompting for multi-document question answering. In *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*, volume 38, pages 19206–19214, 2024.

[8] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*, volume 37, pages 132876–132907, 2024.

[9] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*, volume 37, pages 59532–59569, 2024.

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of Symposium on Operating Systems Principles (SOSP)*, pages 611–626. Association for Computing Machinery, 2023.

[11] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*, volume 37, pages 62557–62583, 2024.

[12] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538. USENIX Association, 2022.

[13] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. FastServe: Iteration-level preemptive scheduling for large language model inference. In *Proceedings of USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 57–74. USENIX Association, 2026.

[14] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.

[15] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–191. USENIX Association, 2024.

[16] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-Seong Chang, and Jiwon Seo. ExeGPT: Constraint-aware resource scheduling for LLM inference. In *Proceedings of International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 369–384. Association for Computing Machinery, 2024.

[17] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with lina. In *Proceedings of USENIX Annual Technical Conference (USENIX ATC)*, pages 945–959. USENIX Association, 2023.

[18] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134. USENIX Association, 2024.

[19] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 193–210. USENIX Association, 2024.

[20] Youpeng Zhao, Di Wu, and Jun Wang. ALISA: Accelerating large language model inference via sparsity-aware KV caching. In *Proceedings of International Symposium on Computer Architecture (ISCA)*, pages 1005–1017. IEEE, 2024.

[21] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 155–172. USENIX Association, 2024.

[22] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338, 2024.

[23] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of ACM Special Interest Group on Data Communication Conference (SIGCOMM)*, pages 38–56. Association for Computing Machinery, 2024.

[24] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*, volume 35, pages 16344–16359, 2022.

[25] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander Smola, and Le Song. Variational reasoning for question answering with knowledge graph. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*, volume 32, 2018.

[26] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2018.

[27] Jeffrey Dalton, Chenyan Xiong, and Jamie Callan. Trec cast 2019: The conversational assistance track overview. *arXiv preprint arXiv:2003.13624*, 2020.

[28] Leslie G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing*, 8(3):410–421, 1979.

[29] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[30] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

[31] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. Subgraph retrieval enhanced model for multi-hop knowledge base question answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[32] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[33] Emman Haider, Daniel Perez-Becker, Thomas Portet, Piyush Madan, Amit Garg, Atabak Ashfaq, David Majercak, Wen Wen, Dongwoo Kim, Ziyi Yang, Jianwen Zhang, Hiteshi Sharma, Blake Bullwinkel, Martin Pouliot, Amanda Minnich, Shiven Chawla, Solianna Herrera, Shahed Warreth, Maggie Engler, Gary Lopez, Nina Chikanov, Raja Sekhar Rao Dheekonda, Bolor-Erdene Jagdagdorj, Roman Lutz, Richard Lundeen, Tori Westerhoff, Pete Bryan, Christian Seifert, Ram Shankar Siva Kumar, Andrew Berkley, and Alex Kessler. Phi-3 safety post-training: Aligning language models with a" break-fix" cycle. *arXiv preprint arXiv:2407.13833*, 2024.

[34] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[35] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[36] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. Turborag: Accelerating retrieval-augmented generation with precomputed kv caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6599–6612, 2025.

[37] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. Epic: Efficient position-independent caching for serving large language models. *arXiv preprint arXiv:2410.15332*, 2024.

[38] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 94–109, 2025.

[39] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. Kvlink: Accelerating large language models via efficient kv cache reuse. *arXiv preprint arXiv:2502.16002*, 2025.

[40] Rui Yang, Haoran Liu, Edison Marrese-Taylor, Qingcheng Zeng, Yuhe Ke, Wanxin Li, Lechao Cheng, Qingyu Chen, James Caverlee, Yutaka Matsuo, and Irene Li. KG-rank: Enhancing large language models for medical QA with knowledge graphs and ranking techniques. In *Proceedings of Workshop on Biomedical Natural Language Processing (BioNLP)*, pages 155–166. Association for Computational Linguistics, 2024.

[41] Dawei Li, Shu Yang, Zhen Tan, Jae Young Baik, Sukwon Yun, Joseph Lee, Aaron Chacko, Bojian Hou, Duy Duong-Tran, Ying Ding, Huan Liu, Li Shen, and Tianlong Chen. DALK: Dynamic co-augmentation of LLMs and KG to answer Alzheimer's disease questions with scientific literature. In

Proceedings of Findings of the Association for Computational Linguistics (EMNLP Findings), pages 2187–2205. Association for Computational Linguistics, 2024.

[42] Yinglian Xie and David O'Hallaron. Locality in search engine queries and its implications for caching. In *Proceedings of IEEE INFOCOM*, volume 3, pages 1238–1247, 2002.

[43] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. BatchLLM: Optimizing large batched LLM inference with global prefix sharing and throughput-oriented token batching. *arXiv preprint arXiv:2412.03594*, 2024.

[44] Alon Talmor and Jonathan Berant. The web as a knowledge-base for answering complex questions. *arXiv preprint arXiv:1803.06643*, 2018.

[45] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10746–10761, 2025. arXiv:2410.05779.

[46] Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. PathRAG: Pruning graph-based retrieval augmented generation with relational paths. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026. arXiv:2502.14902.

[47] You Peng, Ying Zhang, Xuemin Lin, Wenjie Zhang, Lu Qin, and Jingren Zhou. Hop-constrained s-t simple path enumeration: Towards bridging theory and practice. *Proceedings of the VLDB Endowment*, 13(4):463–476, 2019.

[48] Philip Levis, Kun Lin, and Amy Tai. A case against cxl memory pooling. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets)*, pages 18–24, 2023.

[49] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[50] Kexin Chu, Dawei Xiang, Zixu Shen, Yiwei Yang, Zecheng Liu, and Wei Zhang. Dynamic expert quantization for scalable mixture-of-experts inference. *arXiv preprint arXiv:2511.15015*, 2025.

[51] Yuegui Huang, Zhiyuan Fang, Weiqi Luo, Ruoyu Wu, Wuhui Chen, and Zibin Zheng. Dymoe: Dynamic expert orchestration with mixed-precision quantization for efficient moe inference on edge. *arXiv preprint arXiv:2603.19172*, 2026.

[52] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. Moe-apex: An efficient moe inference system with adaptive precision expert offloading. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1185–1200, 2026.

[53] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. Hobbit: A mixed precision expert offloading system for fast moe inference. *arXiv preprint arXiv:2411.01433*, 2024.

[54] Jianing Deng, Song Wang, Dongwei Wang, Zijie Liu, Tianlong Chen, Huanrui Yang, and Jingtong Hu. Gemq: Global expert-level mixed-precision quantization for moe llms. *arXiv preprint arXiv:2605.23078*, 2026.

[55] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E Gonzalez, Matei Zaharia, and Ion Stoica. Moe-lightning: High-throughput moe inference on memory-constrained gpus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 715–730, 2025.

[56] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. Fiddler: Cpu-gpu orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.

[57] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. Daop: Data-aware offloading and predictive pre-calculation for efficient moe inference. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE, 2025.

[58] Yichao Yuan, Lin Ma, and Nishil Talati. Moe-lens: Towards the hardware limit of high-throughput moe llm serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.

[59] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *Proceedings of Machine Learning and Systems*, 7, 2025.

[60] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.

[61] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1018–1031. IEEE, 2024.

[62] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 574–588, 2025.

[63] Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. Taming latency-memory trade-off in moe-based llm serving via fine-grained expert offloading. In *Proceedings of the 21st European Conference on Computer Systems*, pages 176–191, 2026.

[64] Taehyun Kim, Kwanseok Choi, Youngmock Cho, Jaehoon Cho, Hyuk-Jae Lee, and Jaewoong Sim. Monde: Mixture of near-data experts for large-scale sparse models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pages 1–6, 2024.

[65] Haochen Huang, Shuzhang Zhong, Zhe Zhang, Shuangchen Li, Dimin Niu, Hongzhong Zheng, Runsheng Wang, and Meng Li. Hd-moe: Hybrid and dynamic parallelism for mixture-of-expert llms with 3d near-memory processing. *arXiv preprint arXiv:2509.09420*, 2025.

[66] Qi Wu, Chao Fang, Jiayuan Chen, Ye Lin, Yueqi Zhang, Yichuan Bai, Yuan Du, and Li Du. A scheduling framework for efficient moe inference on edge gpu-ndp systems. *arXiv preprint arXiv:2601.03992*, 2026.

[67] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1429–1443. IEEE, 2024.

[68] Lizhou Wu, Haozhe Zhu, Siqi He, Xuanda Lin, Xiaoyang Zeng, and Chixiao Chen. Pimoe: Towards efficient moe transformer deployment on npu-pim system through throttle-aware task offloading. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 1–7. IEEE, 2025.

[69] Yudong Pan, Yintao He, Tianhua Han, Lian Liu, Shixin Zhao, Zhirong Chen, Mengdi Wang, Cangyuan Li, Yinhe Han, and Ying Wang. Trimoe: Augmenting gpu with amx-enabled cpu and dimm-ndp for high-throughput moe inference via offloading. *arXiv preprint arXiv:2603.01058*, 2026.

[70] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Offloading-efficient moe model serving. *arXiv preprint arXiv:2401.14361*, 2025.

[71] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2024.

[72] Akanksha Jain and Calvin Lin. Back to the future: Leveraging belady's algorithm for improved cache replacement. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016.

[73] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *Proceedings of 2024 USENIX annual technical conference (USENIX ATC)*, pages 111–126, 2024.

[74] Byeongho Heo, Song Park, Dongyoon Han, and Sangdoo Yun. Rotary position embedding for vision transformer. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 289–305. Springer, 2024.

[75] Ming Yin, Minshuo Chen, Kaixuan Huang, and Mengdi Wang. A theoretical perspective for speculative decoding algorithm. *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 37:128082–128117, 2024.

[76] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6159–6172, 2024.

[77] Yanyue Xie, Zhi Zhang, Ding Zhou, Cong Xie, Ziang Song, Xin Liu, Yanzhi Wang, Xue Lin, and An Xu. Moe-pruner: Pruning mixture-of-experts large language model using the hints from its router. *arXiv preprint arXiv:2410.12013*, 2024.

[78] Haojie Duanmu, Xiuhong Li, Zhihang Yuan, Size Zheng, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. Mxmoe: Mixed-precision quantization for moe with accuracy and performance co-design. *arXiv preprint arXiv:2505.05799*, 2025.

[79] Wei Huang, Yue Liao, Jianhui Liu, Ruifei He, Haoru Tan, Shiming Zhang, Hongsheng Li, Si Liu, and Xiaojuan Qi. Mixture compressor for mixture-of-experts llms gains more. In *International Conference on Learning Representations*, volume 2025, pages 68628–68648, 2025.

[80] Beichen Huang, Yueming Yuan, Zelei Shao, and Minjia Zhang. Milo: Efficient quantized moe inference with mixture of low-rank compensators. *arXiv preprint arXiv:2504.02658*, 2025.

[81] Fangxin Liu, Ning Yang, Jingkui Yang, Zongwu Wang, Chenyang Guan, Yu Feng, Li Jiang, and Haibing Guan. Earth: An efficient moe accelerator with entropy-aware speculative prefetch and result reuse. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 633–646, 2026.

[82] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. Promoe: Fast moe-based llm serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.

[83] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. Swapmoe: Serving off-the-shelf moe-based large language models with tunable memory budget. *arXiv preprint arXiv:2308.15030*, 2023.

[84] Han Li, Jingwei Sun, Junqing Lin, and Guangzhong Sun. Commitmoe: Efficient fallback-free moe inference with offloading under gpu memory constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 22904–22912, 2026.

[85] Baihui Liu, Kaiyuan Tian, Wei Wang, Zhaoning Zhang, Linbo Qiao, and Dongsheng Li. Alloc-moe: Budget-aware expert activation allocation for efficient mixture-of-experts inference. *arXiv preprint arXiv:2604.08133*, 2026.

[86] Juntong Wu, Jialiang Cheng, Fuyu Lv, Ou Dan, and Li Yuan. Sere: Similarity-based expert re-routing for efficient batch decoding in moe models. *arXiv preprint arXiv:2602.07616*, 2026.

[87] Guoying Zhu, Meng Li, Haipeng Dai, Xuechen Liu, Weijun Wang, Keran Li, Jun Xiao, Ligeng Chen, and Wei Wang. Smoe: An algorithm-system co-design for pushing moe to the edge via expert substitution. In *Proceedings of ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)*, pages 1–17. IEEE, 2026.

[88] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. A scalable processing-in-memory accelerator for parallel graph processing. In *Proceedings of International Symposium on Computer Architecture (ISCA)*, pages 105–117. Association for Computing Machinery, 2015.

[89] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. Graphicionado: A high-performance and energy-efficient accelerator for graph analytics. In *Proceedings of International Symposium on Microarchitecture (MICRO)*, pages 56:1–56:13. IEEE, 2016.

[90] Linghao Song, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. GraphR: Accelerating graph processing using ReRAM. In *Proceedings of International Symposium on High-Performance Computer Architecture (HPCA)*, pages 531–543. IEEE, 2018.

[91] Mingxing Zhang, Youwei Zhuo, Chao Wang, Mingyu Gao, Yongwei Wu, Kang Chen, Christos Kozyrakis, and Xuehai Qian. GraphP: Reducing communication for PIM-based graph processing with efficient data partition. In *Proceedings of International Symposium on High-Performance Computer Architecture (HPCA)*, pages 544–557. IEEE, 2018.

[92] Lifeng Nai, Ramyad Hadidi, Jaewoong Sim, Hyojong Kim, Pranith Kumar, and Hyesoon Kim. GraphPIM: Enabling instruction-level PIM offloading in graph computing frameworks. In *Proceedings of International Symposium on High-Performance Computer Architecture (HPCA)*, pages 457–468. IEEE, 2017.

[93] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. HyGCN: A GCN accelerator with hybrid architecture. In *Proceedings of International Symposium on High-Performance Computer Architecture (HPCA)*, pages 15–29. IEEE, 2020.

[94] Xuhao Chen, Tianhao Huang, Shuotao Xu, Thomas Bourgeat, Chanwoo Chung, and Arvind. FlexMiner: A pattern-aware accelerator for graph pattern mining. In *Proceedings of International Symposium on Computer Architecture (ISCA)*, pages 581–594. IEEE, 2021.

[95] Vidushi Dadu, Sihao Liu, and Tony Nowatzki. PolyGraph: Exposing the value of flexibility for graph processing accelerators. In *Proceedings of International Symposium on Computer Architecture (ISCA)*, pages 595–608. IEEE, 2021.

[96] Quan M. Nguyen and Daniel Sanchez. Fifer: Practical acceleration of irregular applications on reconfigurable architectures. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2021.

[97] Quan M. Nguyen and Daniel Sanchez. Phloem: Automatic acceleration of irregular applications with fine-grain pipeline parallelism. In *Proceedings of the 29th International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1262–1274. IEEE, 2023.

[98] Shilin Tian, Sanjay Gandham, Lingxiang Yin, Mingjie Lin, and Hao Zheng. KAIROS: Fast microarchitectural design space exploration by bridging architectural semantics and logic gates. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Athens, Greece, October 31–November 4 2026. IEEE/ACM.

[99] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. Verilogeval: Evaluating large language models for verilog code generation. In *Proceedings of 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–8. IEEE, 2023.

[100] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. Rtllm: An open-source benchmark for design rtl generation with large language model. In *Proceedings of 2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 722–727. IEEE, 2024.

[101] Payal Chandak, Kexin Huang, and Marinka Zitnik. Building a knowledge graph to enable precision medicine. *Scientific data*, 10(1):67, 2023.

[102] Daniel Scott Himmelstein, Antoine Lizee, Christine Hessler, Leo Brueggeman, Sabrina L Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, and Sergio E Baranzini. Systematic integration of biomedical knowledge prioritizes drugs for repurposing. *elife*, 6:e26726, 2017.

[103] George Tsatsaronis, Georgios Balikas, Prodromos Malakasiotis, Ioannis Partalas, Matthias Zschunke, Michael R Alvers, Dirk Weissenborn, Anastasia Krithara, Sergios Petridis, Dimitris Polychronopoulos, Yannis Almirantis, John Pavlopoulos, Nicolas Baskiotis, Patrick Gallinari, Thierry Artiéres, Axel-Cyrille Ngonga Ngomo, Norman Heino, Eric Gaussier, Liliana Barrio-Alvers, Michael Schroeder, Ion Androutsopoulos, and Georgios Paliouras. An overview of the bioasq large-scale biomedical semantic indexing and question answering competition. *BMC bioinformatics*, 16(1):138, 2015.

[104] Anastasia Krithara, Anastasios Nentidis, Konstantinos Bougiatiotis, and Georgios Paliouras. Bioasq-qa: A manually curated corpus for biomedical question answering. *Scientific data*, 10(1):170, 2023.

[105] Amir Ghazizadeh, Lingxiang Yin, Shilin Tian, Fangzhou Ye, Fan Yao, and Hao Zheng. Rethinking tiling and dataflow for spmm acceleration: A graph transformation framework. In *Proceedings of IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2025.

[106] Jiaqi Yang, Hao Zheng, and Ahmed Louri. Ditile-dgnn: An efficient accelerator for distributed dynamic graph neural network inference. In *Proceedings of International Symposium on Computer Architecture (ISCA)*. ACM, 2025.

[107] Amir Ghazizadeh, Rickard Ewetz, and Hao Zheng. Gamma: Gated multi-hop message passing for homophily-agnostic node representation in gnns. *Advances in Neural Information Processing Systems*, 38:11726–11777, 2026.

[108] Fangzhou Ye, Shilin Tian, Amir Ghazizadeh, and Hao Zheng. Tensorprism: Rethinking sparse high-order tensor acceleration via co-occurrence graph. In *Proceedings of International Symposium on Computer Architecture (ISCA)*. ACM, 2026.

[109] Fangzhou Ye, Lingxiang Yin, and Hao Zheng. Scaling graph neural network training via geometric optimization. In *Proceedings of IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2026.

[110] Fangzhou Ye, Amir Ghazizadeh Ahsaei, and Hao Zheng. Argus: An efficient accelerator design for sparse nonlinear solver. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Athens, Greece, October 31–November 4 2026. IEEE/ACM.

[111] Fangzhou Ye, Wei Zhang, Ahmed Louri, and Hao Zheng. MOSAIC: An accelerator design for heterophilic graph learning. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Athens, Greece, October 31–November 4 2026. IEEE/ACM.

[112] Yixin Xu, Yi Xiao, Zijian Zhao, Franz Müller, Alptekin Vardar, Xiao Gong, Sumitha George, Thomas Kämpfe, Vijaykrishnan Narayanan, and Kai Ni. Embedding security into ferroelectric fet array via in situ memory operation. *Nature Communications*, 14(1):1–8, 2023.

[113] Yiming Chen, Guodong Yin, Hongtao Zhong, Mingyen Lee, Huazhong Yang, Sumitha George, Vijaykrishnan Narayanan, and Xueqing Li. Zebra: A zero-bit robust-accumulation compute-in-memory approach for neural network acceleration utilizing different bitwise patterns. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 153–158, 2024.

[114] Sanwar Ahmed Ovy, Md Ashraful Islam Romel, Yi Xiao, Yixin Xu, Kai Ni, and Sumitha George. Compact Multiplexer Design with Multi-threshold Ferroelectric FETs . In *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 735–739, Los Alamitos, CA, USA, July 2024. IEEE Computer Society.

[115] Binu Christopher, Rwicheek Sarker, Vivek M Bhasi, and Sumitha George. Multi-level cell memory driven efficient cache organization. In *2026 27th International Symposium on Quality Electronic Design (ISQED)*, pages 1–9. IEEE, 2026.